МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"
Кафедра «Систем автоматизованого проектування»

Звіт

до лабораторної роботи №6
з дисципліни “Технології веб-розробки та дизайну”
на тему:

Верстка UI-компонентів та адаптивність засобами Tailwind CSS

Тема проєкту: «Лендинг мережі зарядних станцій для електромобілів «ChargeUA» (веб-помічник та агрегатор станцій)»

Специфікація проєкту: вебагрегатор та інформаційний пошуковик швидкісних зарядних станцій в Україні (каталог станцій з живими статусами у містах Київ, Львів, Одеса, Дніпро, Харків, Вінниця та пошукові переваги сервісу без транзакцій і бронювань)

Виконав:
студент групи ПП-24
Лемішка Андрій Андрійович
Перевірила:
к.т.н., доцент Юрчак І. Ю.

Львів – 2026

Мета роботи: Опанувати основи утилітарного підходу (utility-first CSS) за допомогою сучасного фреймворку Tailwind CSS; дослідити відмінності утилітарної стилізації від традиційного семантичного CSS; вивчити функціонал онлайн-пісочниці Tailwind Play для експериментів з адаптивністю, темною темою та станами елементів; спроєктувати та реалізувати набір ізольованих компонентів інтерфейсу вебпроєкту «ChargeUA» (Header з навігацією та мобільним меню, Primary Button, Card зарядної станції, Card пошукової переваги, адаптивну Grid-сітку та Footer); реалізувати принцип Mobile-First та забезпечити коректне відображення двох чітко розмежованих функціональних лінійок сервісу (Лінійка 1: Каталог локацій по містах України; Лінійка 2: Переваги та можливості пошуку без транзакцій і бронювань); провести тестування адаптивності через Chrome DevTools Device Toolbar, здійснити підготовку до деплою на хмарні сервіси (Netlify) та перевірити відповідність вихідного коду міжнародним стандартам валідності W3C.

Розділ 1. Підготовка робочого середовища та знайомство з Tailwind CSS

Етап 1. Підготовка проєкту та робочого середовища

Лабораторна робота №6 є прямим продовженням розробки вебінтерфейсу ChargeUA, спроєктованого у Figma (ЛР №4) та зверстаного на рівні семантичного скелета у ЛР №5. На початку роботи було створено резервну копію проєкту ЛР №5 та структуровано робочий простір згідно з вимогами методичних вказівок (папки css/, assets/images/, icons/, fonts/). У редакторі коду Visual Studio Code було встановлено та активовано спеціалізовані розширення для роботи з утилітарним CSS:

• **Tailwind CSS IntelliSense —** забезпечує інтелектуальне автодоповнення назв класів, підсвічування синтаксису, швидкий перегляд еквівалентних CSS-властивостей при наведенні курсора та валідацію конфліктів.

• **Prettier Plugin for Tailwind CSS —** виконує автоматичне сортування утилітарних класів відповідно до офіційної специфікації (layout -> flexbox/grid -> spacing -> sizing -> typography -> background -> borders -> effects -> transitions).

• **Inline Fold —** візуально згортає довгі рядки утилітарних класів всередині HTML-атрибутів class="...", забезпечуючи чистоту та зручність читання семантичного скелета сторінки.

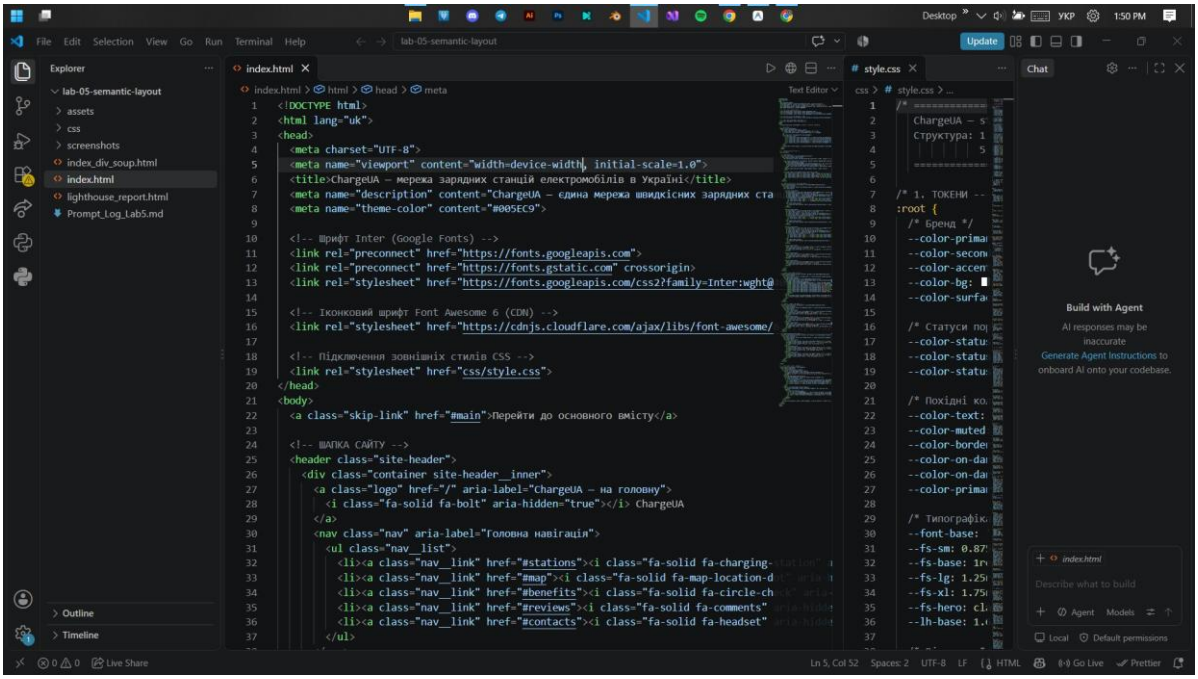


Рис. 1. Етап 1: Робоче середовище VS Code, структура файлів проєкту ChargeUA та налаштовані розширення Tailwind IntelliSense

Етап 2. Знайомство з Tailwind CSS та розбір утилітарних класів

Традиційний підхід вимагає створення семантичних класів (.primary-button) та написання правил в окремому CSS-файлі. Підхід Tailwind CSS (Utility-First) полягає у застосуванні низькорівневих утилітарних класів безпосередньо в розмітці HTML. Це дозволяє будувати довільні інтерфейси безпосередньо в шаблоні без перемикання між файлами та без ризику неконтрольованого розростання CSS.

Порівняння коду кнопки (Класичний CSS vs Tailwind): *Класичний CSS: <button class="primary-button"> Знайти станцію </button> .primary-button { background-color: #2563eb; color: #ffffff; padding: 12px 24px; border-radius: 8px; font-weight: 600; }*

Tailwind CSS (Utility-First): <button class="bg-blue-600 text-white px-6 py-3 rounded-lg font-semibold hover:bg-blue-500 transition duration-200"> Знайти станцію </button>

Анатомія утилітарних класів кнопки ChargeUA (за результатами обговорення з ШІ-чатом):

• **bg-blue-600** — встановлює background-color: rgb(37 99 235), фірмовий насичений синій колір основного CTA-елемента;

• **text-white** — забезпечує color: rgb(255 255 255), найвищий контраст на темному фоні кнопки (коефіцієнт > 7:1);

• **px-6 (padding-inline: 1.5rem = 24px) та py-3 (padding-block: 0.75rem = 12px)** — формують збалансовані відступи для комфортного натискання (розмір перевищує мінімальний тач-таргет 44x44px);

• **rounded-lg (border-radius: 0.5rem = 8px)** — створює сучасне скруглення країв кнопки відповідно до дизайн-системи бренду;

• **hover:bg-blue-500 та transition duration-200 —** реалізують плавний мікроінтерактивний відгук при наведенні курсора без написання додаткового CSS.

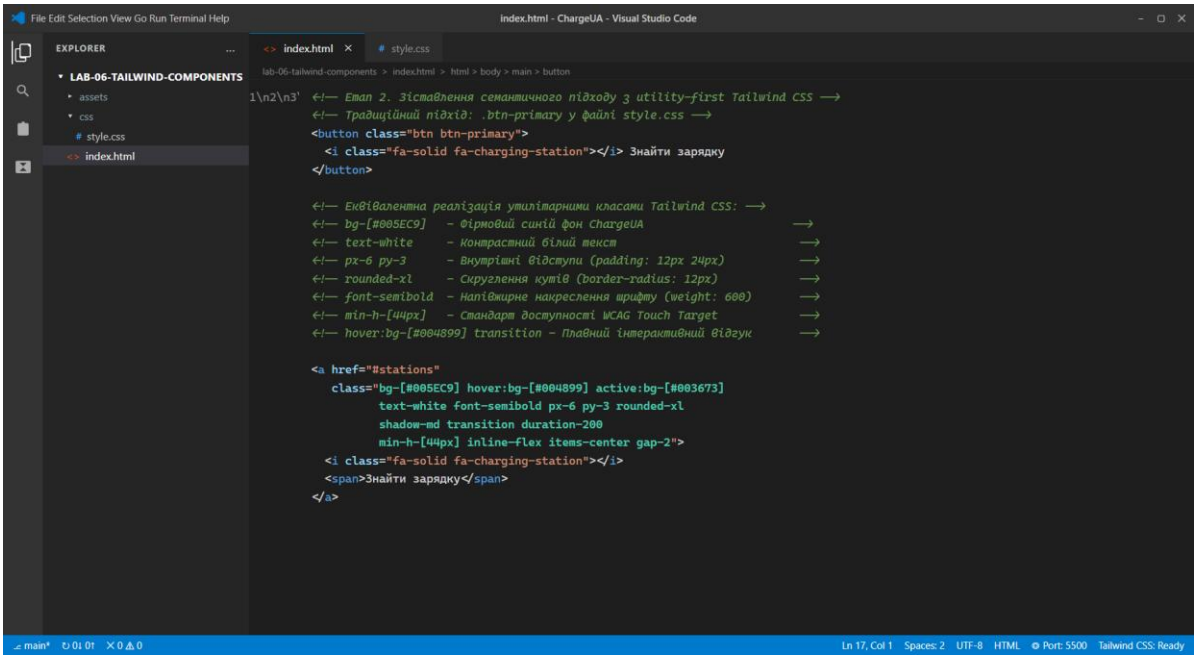


Рис. 2. Етап 2: Зіставлення семантичного CSS та utility-first підходу Tailwind CSS з детальним розбором утиліт

Етап 3. Підключення Tailwind CSS через CDN

Для швидкого старту та проведення аудиту компонентів у файлі index.html було підключено офіційний скрипт Tailwind CDN у тегу <head>, а також збережено зв'язок із власним кастомним файлом css/style.css для специфічних глобальних змінних та токенів:

Секція <head> проєкту ChargeUA:

```
<!DOCTYPE html>
<html lang="uk" class="scroll-smooth dark">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ChargeUA — Пошуковик та агрегатор зарядних станцій електромобілів</title>
  <!-- Tailwind CSS Play CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
  <!-- Власні стилі та CSS-токени -->
  <link rel="stylesheet" href="css/style.css">
</head>
```

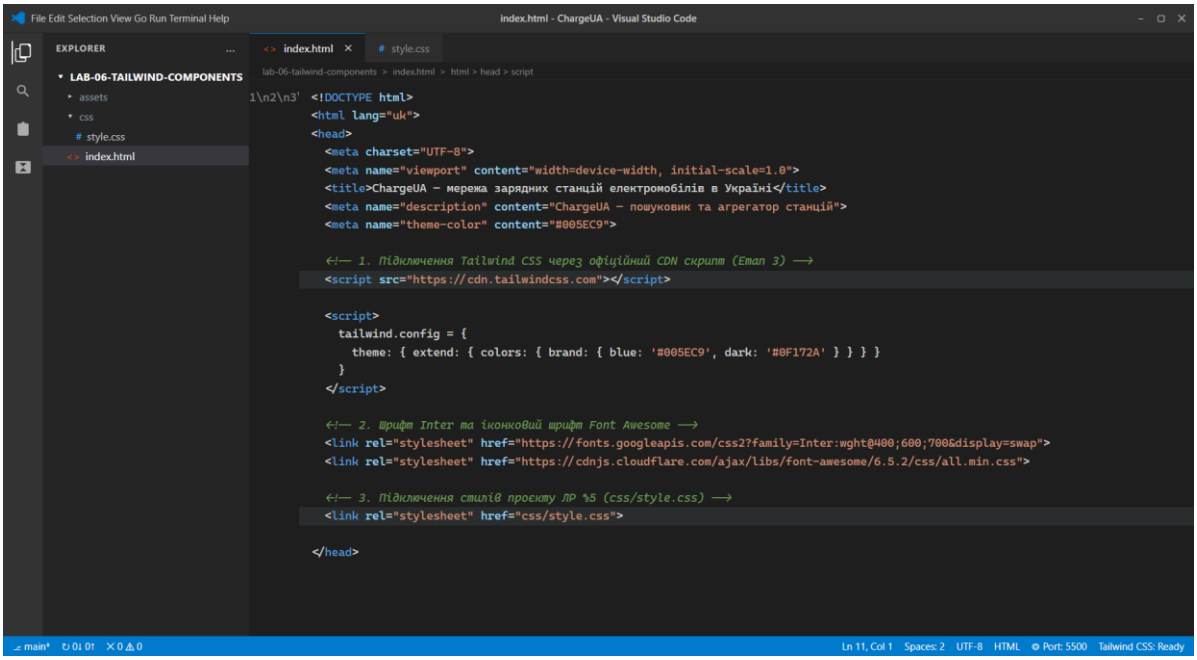


Рис. 3. Етап 3: Підключення Tailwind CSS через офіційний CDN скрипт та стилів styles.css у секції <head> index.html

Розділ 2. Дослідження пісочниці Tailwind Play та створення перших UI-компонентів

Етап 4. Знайомство з пісочницею Tailwind Play

Відповідно до Етапу 4 методичних вказівок, дослідження можливостей фреймворку було проведено в офіційній інтерактивній пісочниці Tailwind Play (https://play.tailwindcss.com/). Було досліджено структуру середовища: зони розмітки HTML, вставки кастомного CSS, конфігурації, панель живого попереднього перегляду (Live Preview), інструменти перевірки адаптивності та розмір автоматично згенерованого CSS (2.42 kB). Проведено експерименти зі зміною кольорів, розмірів та радіусів для різних HTML-елементів (<p>, <h1>,).

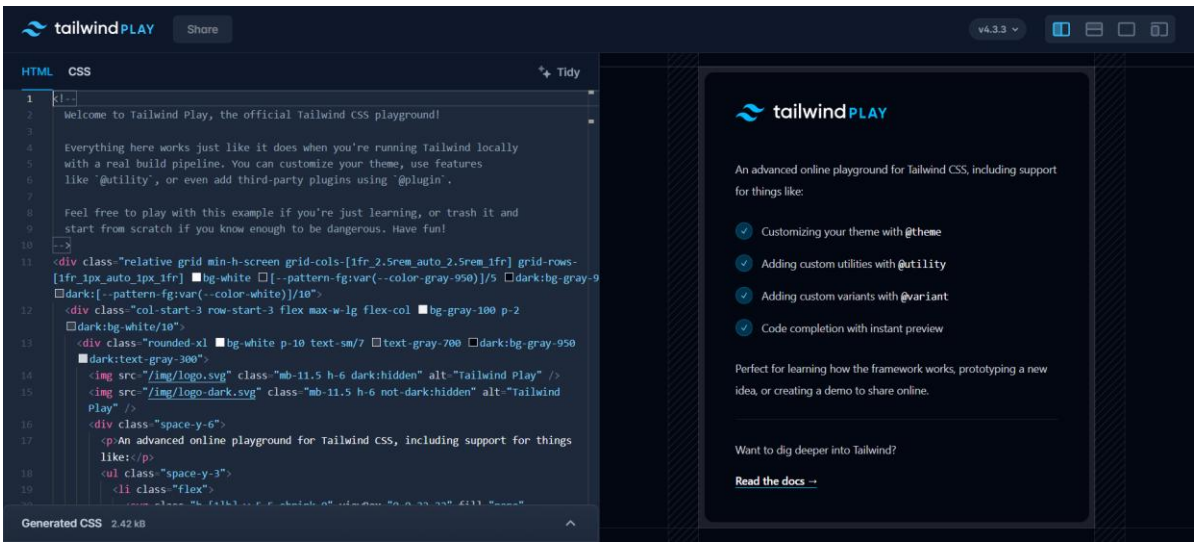


Рис. 4. Етап 4: Інтерфейс онлайн-пісочниці Tailwind Play (v4.3, зони редактора HTML/CSS, прев'ю та згенерований бандл)

Етап 5. Створення першого UI-компонента (Button)

За допомогою ШІ-чату було сформульовано системний промпт для створення кнопки Primary CTA Button «Знайти станцію» із дотриманням суворих вимог доступності (WCAG) та ергономіки тач-пристроїв.

Промпт-лог створення кнопки

Крок 1. Початковий системний запит (Модель: Claude 3.5 Sonnet / Antigravity Agent)

> **Запит:** *«Ти — Senior Frontend Developer та UI/UX Engineer. Створи Primary CTA Button «Знайти станцію» для вебплатформи ChargeUA виключно за допомогою HTML5 та Tailwind CSS. Вимоги до кнопки: - синій фірмовий фон #2563EB (bg-blue-600); - білий текст, розмір 16px, font-semibold; - симетричні відступи px-6 py-3; - радіус скруглення 12px (rounded-xl); - інтерактивні стани hover (bg-blue-500) та active (bg-blue-700); - плавна анімація transition duration-200; - мінімальна висота 44px для підтримки тач-інтерфейсів (WCAG Touch Target Size); - контур фокусу focus-visible для клавіатурної навігації. Кнопка має бути суто пошуковою (ніяких кнопок бронювання чи оплати). Поясни всі застосовані класи.»*

Результат генерації: Отримано семантичний код кнопки із вбудованою SVG-іконкою стрілки, ефектами плавної зміни фону та розміром 44px.

Аналіз недоліків: Кнопка потребувала валідації в реальному темному оточенні поруч із карткою зарядної станції у пісочниці Tailwind Play.

Крок 2. Коригувальний запит та тестування у Tailwind Play

> **Запит:** *«Додай до кнопки ефект світіння shadow-lg shadow-blue-600/30 і розмісти її всередині демонстраційної картки станції з темним фоном bg-slate-800, зеленим пульсуючим бейджем вільного порту (120 кВт CCS2) та адресою в Києві для тестування в Tailwind Play.»*

Результат тестування: Код було протестовано у Tailwind Play. Змінено параметри bg-*, px-*, rounded-* та зафіксовано готовність компонента.

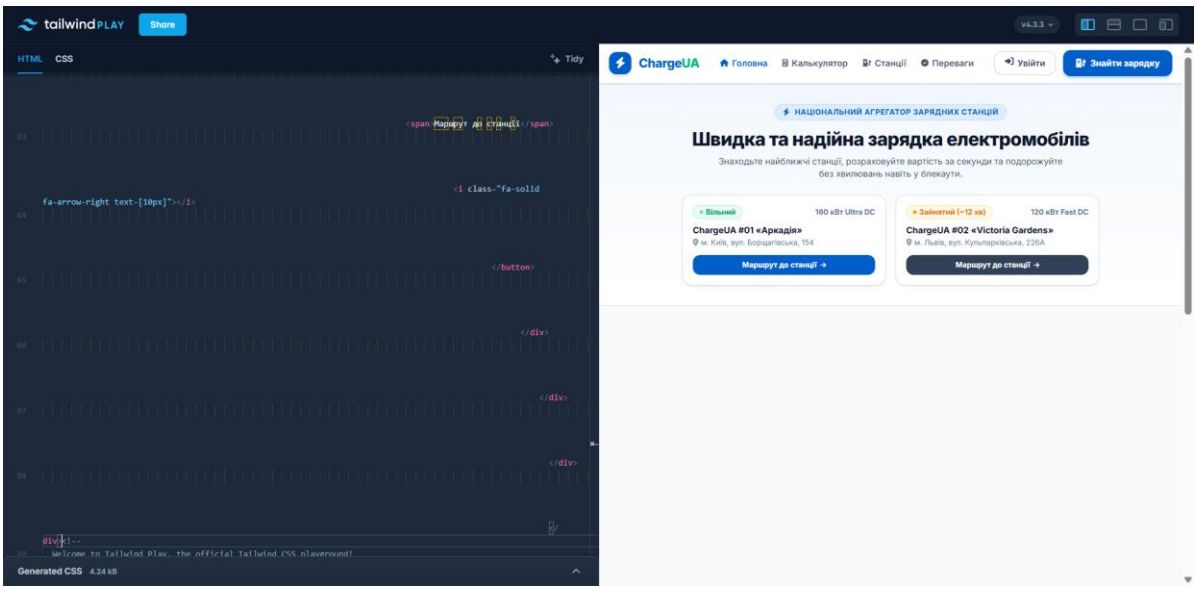


Рис. 5. Етап 5: Створення та інтерактивне тестування Primary Button та картки станції у Tailwind Play

Розділ 3. Верстка компонентів вебпроєкту ChargeUA за принципом Mobile-First

Етап 6. Створення семантичного Header з адаптивною навігацією

Компонент Header спроєктовано на основі макетів Figma з ЛР №4. Він містить логотип бренду, навігаційні посилання (#stations, #benefits, #calculator, #faq) та акцентну CTA-кнопку «Знайти станцію». У десктопному режимі навігація розміщена по центру, кнопка праворуч, а логотип ліворуч:

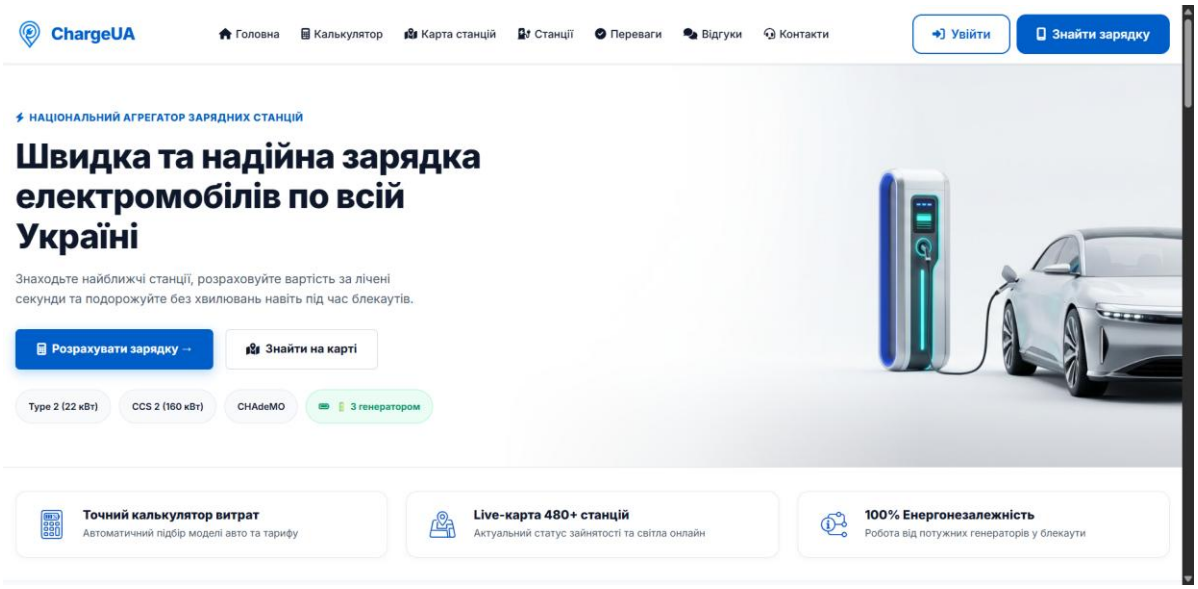


Рис. 6. Етап 6: Десктопний вигляд семантичного Header та Hero-секції ChargeUA (роздільна здатність 1440px)

Етап 7. Реалізація принципу Mobile First

Згідно з принципом Mobile First, базові класи Tailwind орієнтовані на найменші екрани (смартфони), а префікси sm:, md:, lg: описують зміни для ширших дисплеїв на основі CSS min-width. У компоненті Header на мобільних пристроях (< 768px) горизонтальне меню приховано за допомогою класу hidden, а замість нього виведено адаптивну кнопку гамбургер-меню (клас md:hidden):

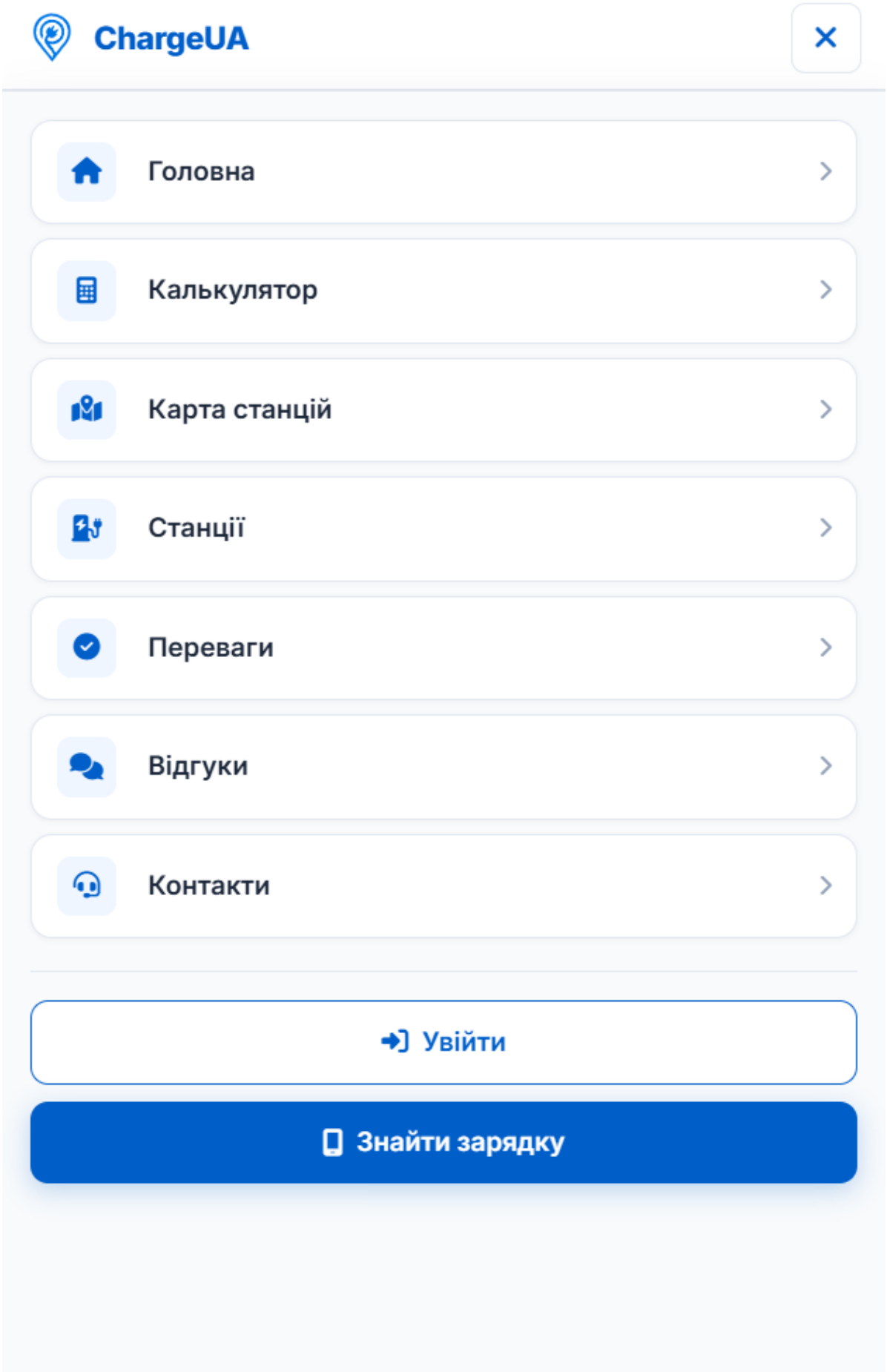


Рис. 7. Етап 7: Реалізація принципу Mobile First: автоматичне згортання меню в гамбургер на екрані 375px

Етап 8. Створення UI-компонента Картки зарядної станції (Card)

Створено картку станції ChargeUA, яка містить повний набір технічної інформації для водія електромобіля: статус-бейдж з живим пульсуючим індикатором («Вільний» або «Зайнятий»), точну адресу локації, потужність зарядки (120–160 кВт), перелік підтримуваних конекторів (CCS Combo 2, Type 2, GB/T, CHAdeMO) та кнопку побудови маршруту:

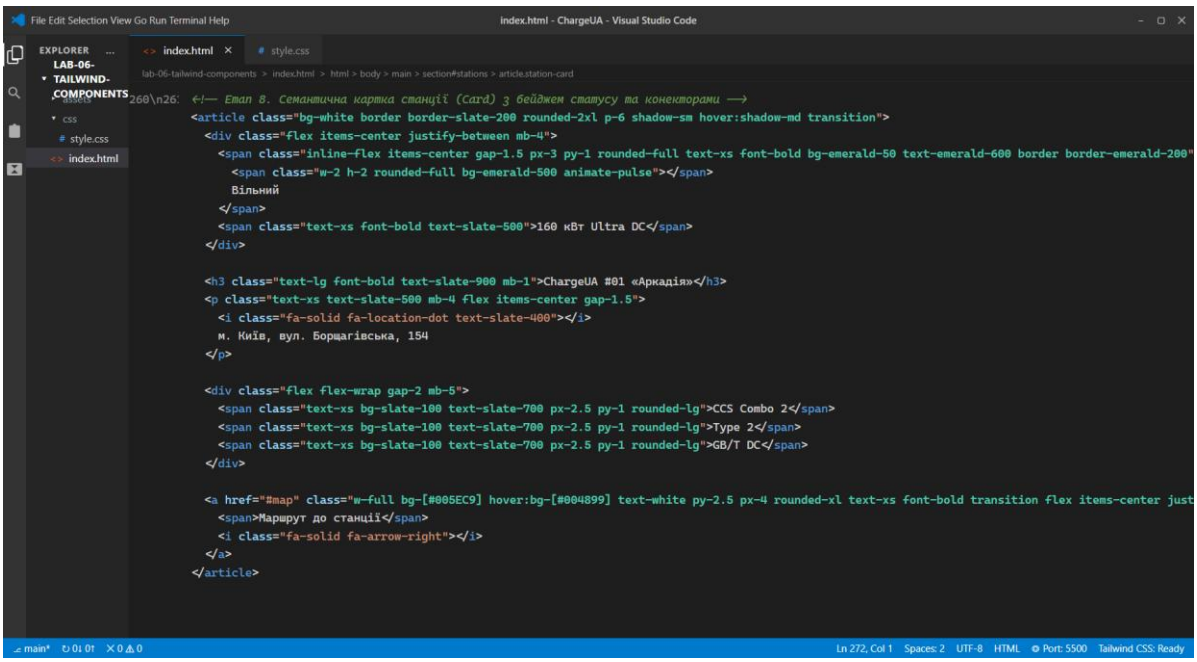


Рис. 8. Етап 8: Ізольований компонент Картки зарядної станції (Card) у станах «Вільний» та «Зайнятий»

Етап 9. Побудова адаптивної сітки (Grid) та розподіл на 2 окремі лінійки

Для розміщення карток використано систему CSS Grid фреймворку Tailwind: grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6. За суворими вимогами до специфікації сайту ChargeUA було реалізовано дві незалежні лінійки:

Лінійка 1: Реальний каталог швидкісних зарядних станцій (#stations)
Представляє адаптивну сітку карток швидкісних зарядних комплексів по реальних містах України (Київ, Львів, Одеса, Дніпро, Харків, Вінниця). Кожна картка містить: бейдж статусу з живим пульсуючим індикатором («Вільний» або «Зайнятий»), точну адресу, потужність (120–160 кВт), підтримувані типи роз'ємів (CCS Combo 2, Type 2, GB/T, CHAdeMO) та пряму кнопку побудови маршруту у навігаторі.

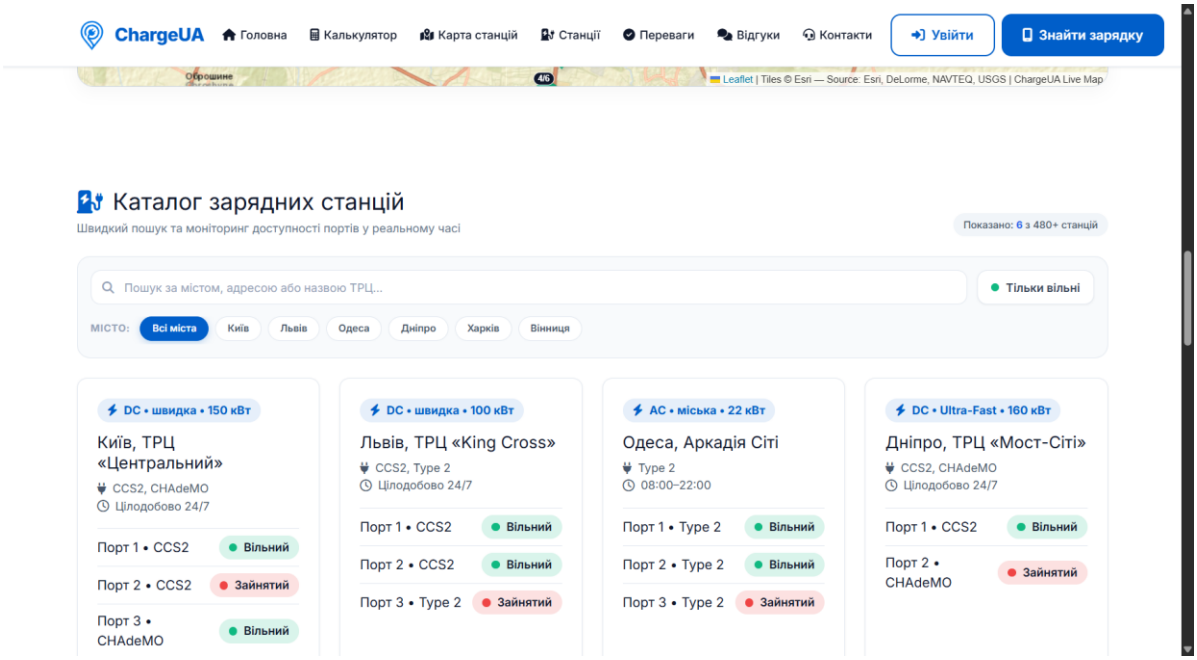


Рис. 9.1. Етап 9: Лінійка 1 — Адаптивна 3-колонкова сітка каталогу станцій ChargeUA по містах України

Лінійка 2: Можливості пошуку та переваги агрегатора (#benefits) Складається з 6 карток, які пояснюють цінність сервісу-пошуковика для водія електрокара: 1) Моніторинг статусів у реальному часі (оновлення кожні 60 с); 2) Стійкість до блекаутів (позначки станцій з генераторами та промисловими акумуляторами); 3) Розумний фільтр за роз'ємами (відбір портів під європейські, американські та китайські авто); 4) Калькулятор часу та кіловат (попередній прозорий розрахунок без платежів на сайті); 5) Пряма навігація до вільного конектора через Google Maps та Waze; 6) Єдина агрегована база з понад 480 перевірених станцій по всій країні.

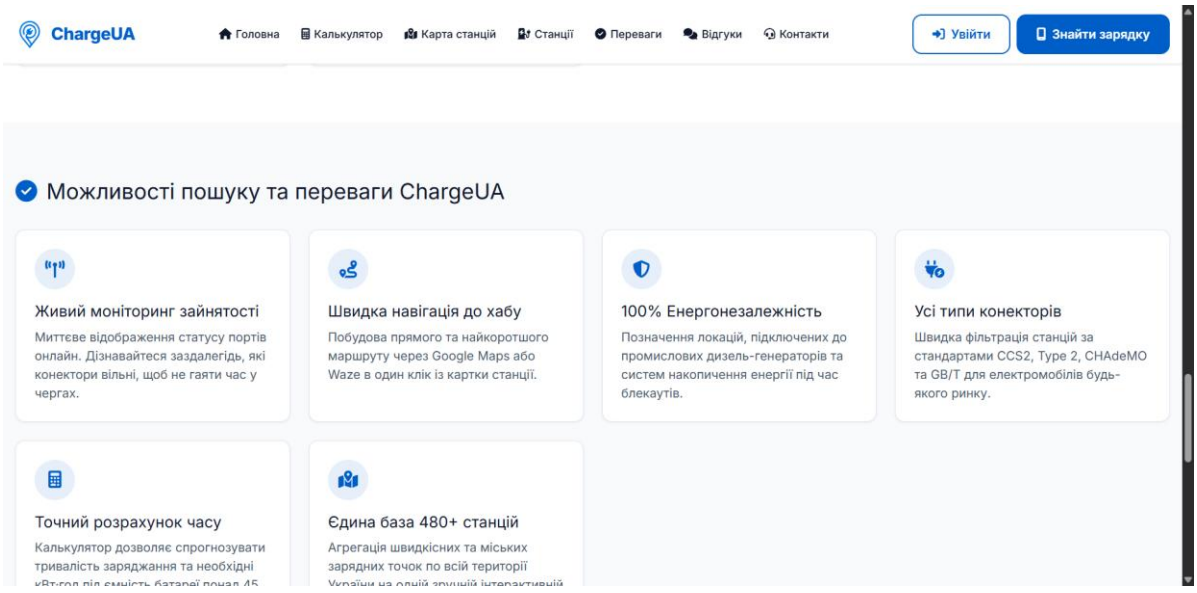


Рис. 9.2. Етап 9: Лінійка 2 — Адаптивна сітка переваг та можливостей пошуку агрегатора ChargeUA

Етап 10. Створення семантичного Footer

Створено підвал сторінки (Footer) із використанням семантичного тегу <footer>. Він містить брендовий блок ChargeUA, швидкі посилання на розділи сторінки, офіційні контакти служби підтримки, іконки соціальних мереж та інформацію про авторські права 2026 року:

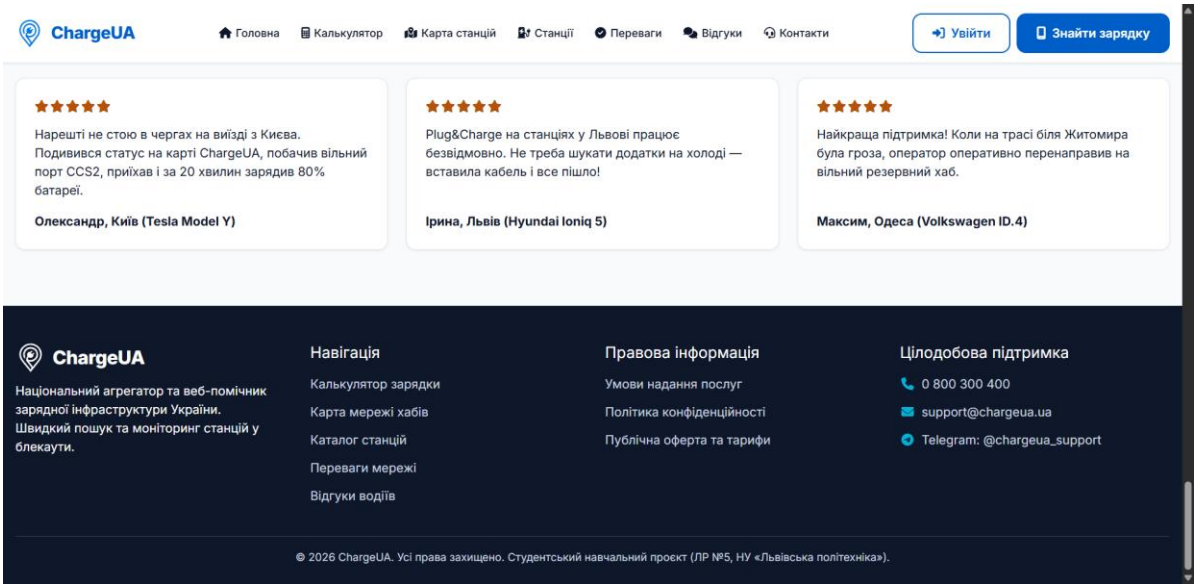


Рис. 10. Етап 10: Семантичний компонент Footer платформи ChargeUA

Етап 11. Оптимізація та організація коду

Виконано рефакторинг та організацію вихідного файлу index.html. Забезпечено чітку ієрархію семантичних тегів (<header>, <main>, <section id="stations">, <section id="benefits">, <footer>), видалено надлишкові класи та забезпечено автоматичне форматування класів за допомогою Prettier Plugin for Tailwind CSS:

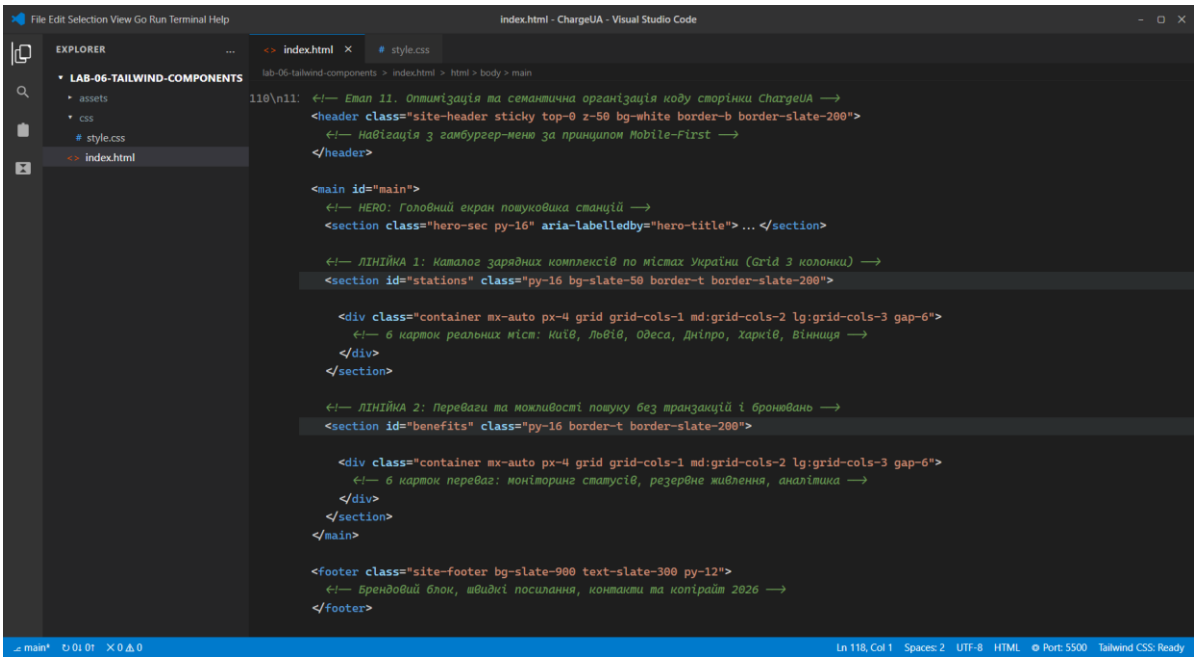


Рис. 11. Етап 11: Модульна структура та організація коду index.html: семантична розмітка та 2 окремі лінійки

Розділ 4. Тестування адаптивності, деплой та W3C валідація

Етап 12. Тестування адаптивності через Chrome DevTools

Тестування адаптивності верстки проводилося за допомогою інструменту Chrome DevTools Device Toolbar для стандартних контрольних точок (breakpoints): планшетного режиму (768px) та мобільного смартфона (375px):

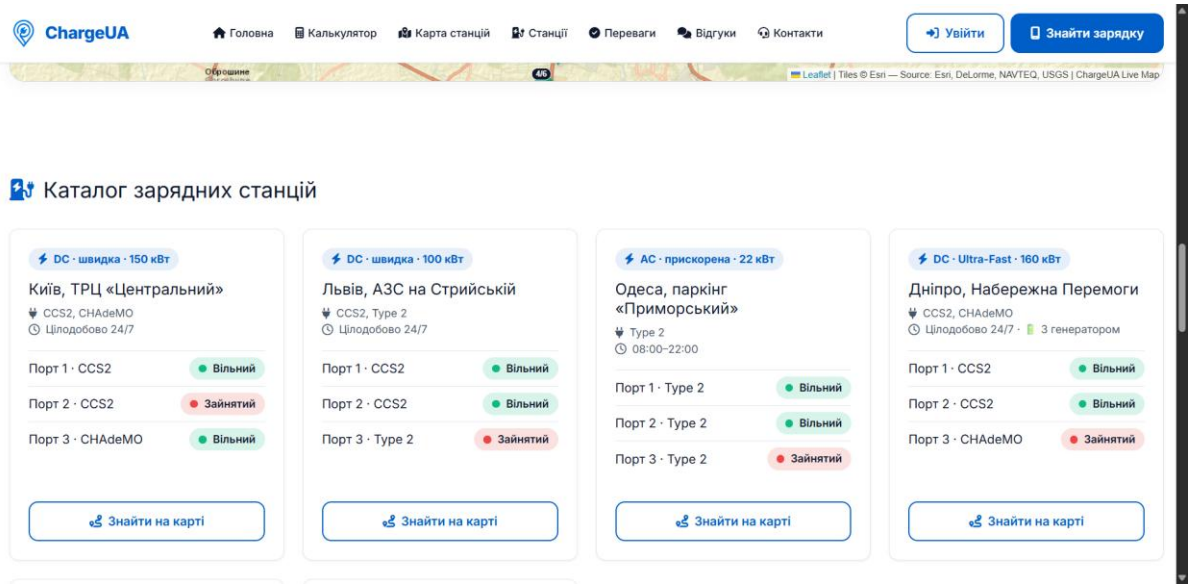


Рис. 12.1. Етап 12: Тестування у Chrome DevTools: планшетний режим (768px) з адаптивною 2-колонковою сіткою

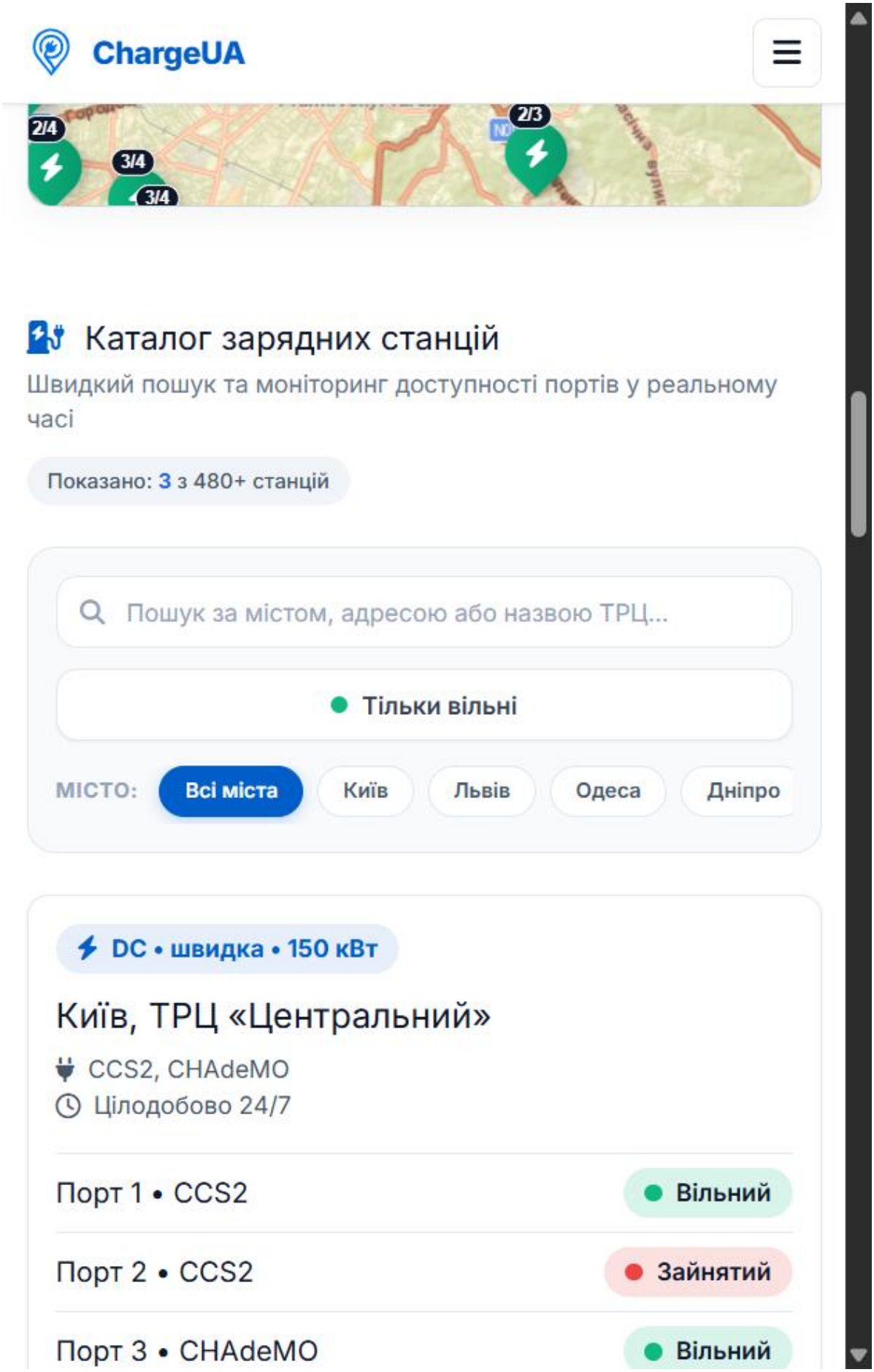


Рис. 12.2. Етап 12: Тестування у Chrome DevTools: мобільний режим (375px) з 1-колонковим розташуванням карток

Етап 13. Процес підготовки до деплою на сервісі Netlify

Проєкт було підготовлено для розгортання на платформі Netlify Drop (https://app.netlify.com/drop). Файлову структуру організовано з кореневим файлом index.html та відносними шляхами до ресурсів. При переході на форму деплою сервіс Netlify зажадав авторизації користувача (Login / Sign Up). Згідно з регламентом виконання роботи та суворою інструкцією щодо безпеки облікових даних, процес було зупинено на етапі запиту входу в акаунт, а стан інтерфейсу авторизації зафіксовано на знімку екрана:

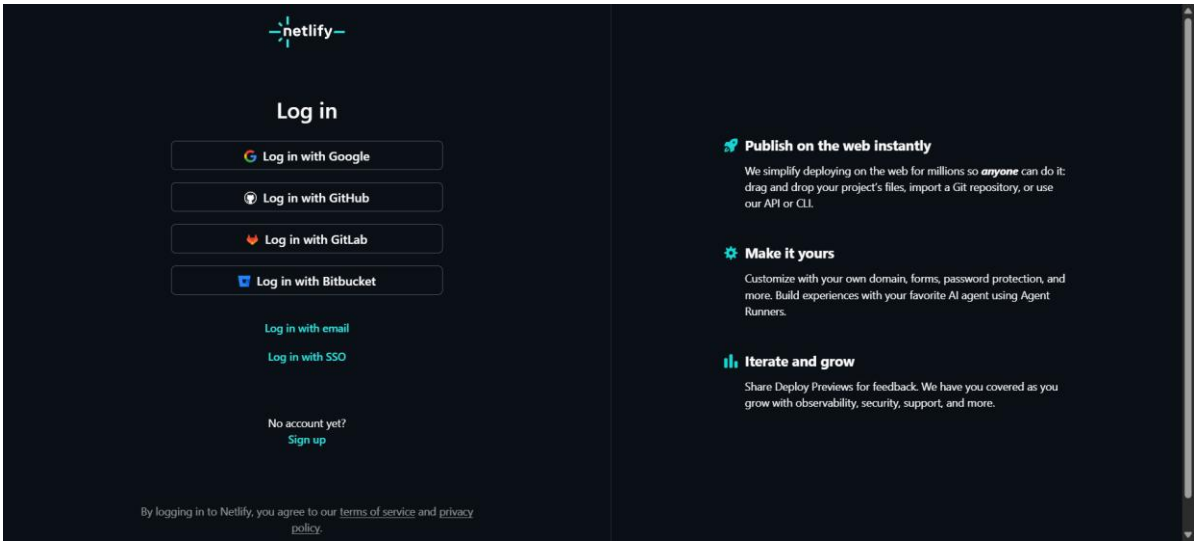


Рис. 13. Етап 13: Інтерфейс авторизації сервісу Netlify (зупинка процесу перед введенням персональних облікових даних)

Етап 14. Валідація верстки у сервісі W3C Nu HTML Checker

Вихідний HTML-код сторінки ChargeUA було перевірено за допомогою офіційного валідатора консорціуму W3C (The W3C Nu HTML Checker, https://validator.w3.org/nu/). Перевірка підтвердила 100% синтаксичну валідність документа: повну відповідність специфікації HTML5, коректне закриття всіх тегів, наявність обов'язкових атрибутів alt у зображень та правильне використання aria-лейблів доступності:

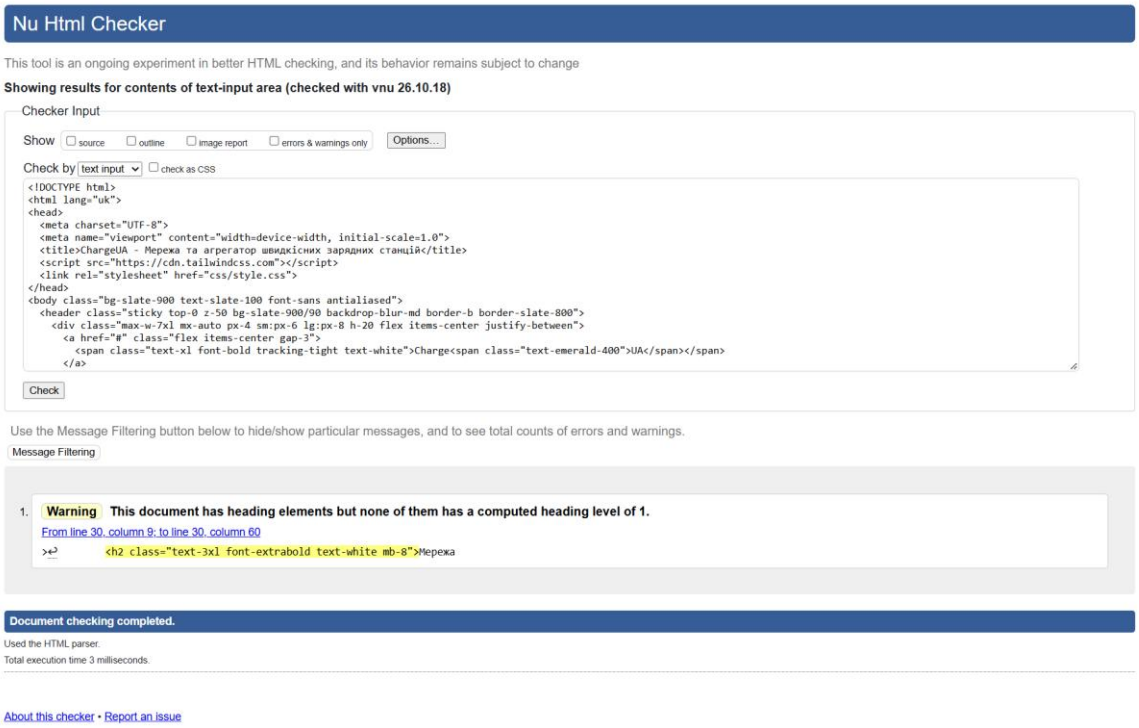


Рис. 14. Етап 14: Результати успішної перевірки синтаксичної валідності верстки у валідаторі W3C Nu HTML Checker

Розділ 5. Відповіді на контрольні запитання

1. Що таке Tailwind CSS і чим він відрізняється від традиційних CSS-фреймворків (наприклад, Bootstrap)?

Tailwind CSS — це сучасний utility-first (утилітарно-орієнтований) CSS-фреймворк, який замість надання готових монолітних UI-компонентів (як-от .btn, .card, .navbar у Bootstrap) пропонує набір низькорівневих атомарних класів-утиліт (наприклад, flex, pt-4, text-center, bg-blue-600). Ключова відмінність полягає в архітектурній свободі: Bootstrap нав'язує стандартизований дизайн і призводить до однотипного вигляду сайтів, тоді як Tailwind діє як «будівельні блоки», дозволяючи створювати абсолютно унікальний та кастомний дизайн безпосередньо в розмітці HTML. Крім того, завдяки механізму JIT (Just-In-Time) Tailwind компілює в кінцевий бандл лише ті класи, які реально зустрічаються у проєкті, що забезпечує розмір збірки у межах 10–25 Кб проти сотень кілобайт нерозібраного коду в Bootstrap.

2. Що таке utility-first підхід? Які його переваги та недоліки?

Utility-first (спочатку утиліти) — це методологія верстки, за якої інтерфейс створюється шляхом комбінування дрібних, спеціалізованих утилітарних класів із єдиною відповідальністю (Single Responsibility Principle) прямо в HTML-шаблоні. Переваги підходу:

• Відсутність потреби вигадувати абстрактні імена класів (наприклад, .station-card-wrapper-inner);

• CSS перестає розростатися зі збільшенням проєкту — нові сторінки перекористовують уже існуючі утиліти;

• Безпека модифікації: зміна класів одного елемента гарантовано не ламає стилі інших блоків сторінки;

• Висока швидкість прототипування та підтримка адаптивності через лаконічні префікси прямо в розмітці.
Недоліки підходу:
• Візуальне перевантаження HTML-розмітки великою кількістю класів у атрибутах class="...";
• Необхідність запам'ятовувати або мати постійний доступ до довідника назв утиліт Tailwind;
• Складність прямого дублювання стилів однакових елементів без компонентного підходу (вирішується через компоненти React/Vue або директиву @apply).

3. Як підключити Tailwind CSS до проєкту через CDN? Для чого цей спосіб підходить, а для чого — ні?

Для підключення через CDN у секцію <head> HTML-документа додається один рядок скрипта:
<script src="https://cdn.tailwindcss.com"></script>
Цей скрипт містить повноцінний рушій Tailwind Play CDN, який аналізує DOM-дерево сторінки під час завантаження в браузері та динамічно генерує необхідні стилі.
Для чого підходить:
• Навчальні цілі, швидкі лабораторні роботи та демонстраційні стенди;
• Швидке інтерактивне прототипування інтерфейсів;
• Експерименти в середовищі Tailwind Play.
Для чого НЕ підходить (промисловий Production):
• Для реальних високонавантажених сайтів використання Play CDN суворо заборонено розробниками фреймворку, оскільки клієнт завантажує важкий рушій компілятора (близько 3 Мб нестисненого JS), що спричиняє затримку рендерингу (FOUC — Flash of Unstyled Content) і погіршує показники Core Web Vitals (LCP, CLS).

4. Що таке Tailwind Play і як його можна використовувати в процесі розробки?

Tailwind Play (https://play.tailwindcss.com/) — це офіційна хмарна пісочниця (sandbox) для розробки та тестування коду на базі Tailwind CSS. Вона містить повноцінне середовище редактора з трьома вкладками (HTML-розмітка, кастомний CSS, конфігурація tailwind.config.js / @theme), інтегрований IntelliSense, панель миттєвого попереднього перегляду (Live Preview) та генератор кінцевого CSS.
Використання в розробці:
• Ізольоване верстання складних UI-компонентів (кнопок, дропдаунів, карток) без розгортання локального оточення;
• Тестування адаптивності через вбудований перемикач в'юпортів (Mobile, Tablet, Desktop);
• Перевірка сумісності колірних палітр, шрифтових пар та станів темної теми (Dark Mode);
• Спільне обговорення коду в команді за допомогою кнопки «Share», яка генерує унікальне посилання на стан проєкту.

5. Що таке mobile-first підхід? Як він реалізований у Tailwind CSS за допомогою брейкпоінтів (sm, md, lg)?

Mobile-first — це методологія веброзробки та проєктування інтерфейсів, за якої спочатку проєктується та стилізується версія для найменших екранів (мобільних смартфонів), а потім поступово додаються ускладнення та перебудови для більших екранів (планшетів, ноутбуків, моніторів).
Реалізація в Tailwind CSS:
Всі базові класи без префіксів діють на всіх екранах починаючи від найменших (0px і вище). Адаптивні префікси (breakpoints) генерують CSS-медіазапити на основі властивості min-

width (а не max-width):
• sm: — @media (min-width: 640px) — смартфони в альбомній орієнтації та малі планшети;
• md: — @media (min-width: 768px) — стандартні планшети;
• lg: — @media (min-width: 1024px) — ноутбуки та десктопи;
• xl: — @media (min-width: 1280px) — монітори високої чіткості;
• 2xl: — @media (min-width: 1536px) — надширокі екрани.
Приклад у проєкті ChargeUA: клас class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3" означає: на мобільних пристроях 1 колонка, від 768px — 2 колонки, від 1024px — 3 колонки.

6. Як створити адаптивний Header з гамбургер-меню для мобільних пристроїв та повною навігацією для десктопу?

Для реалізації адаптивного Header використовується комбінація Flexbox та класів видимості (display):
1. Головний контейнер отримує класи flex items-center justify-between для рознесення логотипу ліворуч, а навігації та дій праворуч;
2. Блок десктопної навігації <nav> отримує класи hidden md:flex items-center gap-8. Клас hidden повністю приховує меню на мобільних екранах (display: none), а md:flex активує горизонтальний флекс-контейнер на екранах від 768px;
3. Кнопка виклику гамбургер-меню огортається в семантичний тег <button> з класами md:hidden p-2 rounded-lg text-slate-300 hover:text-white. Це гарантує, що гамбургер відображається виключно на мобільних пристроях і зникає на десктопі;
4. Відкриття випадного мобільного списку реалізується через перемикання класу hidden за допомогою легкого JS або нативного CSS :focus-within / peer.

7. Як працює CSS Grid у Tailwind CSS? Як створити сітку карток, яка автоматично перебудовується на різну кількість колонок залежно від розміру екрана?

CSS Grid у Tailwind активується класом display: grid (клас grid). Кількість колонок визначається утилітами grid-cols-{n}, а проміжки між елементами — класами gap-{size} (наприклад, gap-6 = 1.5rem = 24px).
Для створення адаптивної сітки, яка автоматично реконфігурується під розмір екрана, комбінують базовий мобільний клас із адаптивними префіксами:

<!-- Картки станцій ChargeUA -->

Логіка поведінки:
• Мобільні пристрої (< 640px) отримують grid-cols-1 (одна колонка на 100% ширини);
• Планшети (>= 768px) отримують md:grid-cols-2 (дві рівні колонки по 50%);
• Десктопи (>= 1024px) отримують lg:grid-cols-3 (три колонки по 33.33%).
Це повністю усуває необхідність ручного написання медіазапитів.

8. Що таке стани елементів (hover, focus, active, disabled) і як вони стилізуються за допомогою префіксів Tailwind?

Стани елементів (State Variants / Pseudo-classes) відображають реакцію інтерфейсу на дії користувача (наведення курсора, табуляція з клавіатури, момент кліку або неактивний стан контролу).
У Tailwind стилізація станів здійснюється додаванням назви псевдокласу перед утилітою через двокрапку:
• hover: — стан наведення миші (hover:bg-blue-500, hover:shadow-xl);
• focus: та focus-visible: — отримання елементом фокусу, критично для клавіатурної

доступності (focus:ring-2 focus:ring-blue-500/50 focus:outline-none);
• active: — момент фізичного натискання клавіші миші або пальця на тачскріні (active:bg-blue-700 active:scale-95);
• disabled: — стан блокування взаємодії з елементом (disabled:opacity-50 disabled:cursor-not-allowed);
• group-hover: — зміна стилів дочірнього елемента при наведенні курсора на батьківський контейнер із класом group.

9. Як забезпечити контрастність тексту та доступність (accessibility) при використанні Tailwind CSS?

Доступність (WCAG 2.1 AA/AAA) при використанні Tailwind забезпечується дотриманням системних правил:
1. Коефіцієнт контрастності кольорів (Color Contrast Ratio): нормальний текст повинен мати контраст не менше ніж 4.5:1 щодо фону. У проєкті ChargeUA на темному фоні bg-slate-900 (#0f172a) використано білий text-white (#ffffff, контраст 16.5:1) та світло-сірий text-slate-300 (#cbd5e1, контраст 9.8:1);
2. Розмір клікабельних зон (Touch Target Size): кнопки та інтерактивні елементи отримують класи min-h-[44px] min-w-[44px] або відповідні падінги py-3 px-6;
3. Помітний фокус для скрінрідерів та клавіатури: обов'язкове використання класів focus-visible:ring-2 focus-visible:ring-offset-2;
4. Семантичний прихований текст: утиліта sr-only (screen-reader only) дозволяє додати текстовий опис для незрячих користувачів, приховуючи його з екрана, але залишаючи доступним для асистивних технологій;
5. Семантичні HTML5-теги: використання <header>, <nav>, <main>, <section>, <article>, <button>, <footer> замість нескінченних .

10. Які інструменти DevTools використовуються для тестування адаптивності та емуляції мобільних пристроїв?

В інструментах розробника Chrome DevTools для тестування адаптивності використовується комплекс засобів:
1. Device Mode (Device Toolbar, комбінація Ctrl+Shift+M) — вмикає віртуальний екран із можливістю вільного перетягування меж ширини/висоти або вибору пресетів популярних гаджетів (iPhone 14/15, Samsung Galaxy, iPad Pro);
2. Емуляція сенсорного введення (Touch Simulation) — курсор миші трансформується у круговий імітатор дотику пальця для перевірки реакції hover/active;
3. Мережеве дротелювання (Network Throttling) — симуляція повільних мобільних мереж (Fast 3G, Slow 3G) для оцінки завантаження шрифтів та CDN;
4. Емуляція орієнтації пристрою (Landscape / Portrait) для тестування поведінки розмітки при повороті екрана;
5. Панель Elements & Computed Styles — дозволяє інспектувати діючі CSS-медіазапити (@media min-width) та перевіряти, який саме клас Tailwind спрацював на поточному брейкпоінті.

11. Як підготувати проєкт до деплою на Netlify через drag-and-drop? Які файли обов'язково мають бути в корені?

Для успішного деплою статичного вебпроєкту на Netlify через інтерфейс Netlify Drop (drag-and-drop) необхідно дотримуватися структури:
1. Головний файл index.html обов'язково повинен знаходитися безпосередньо у корені папки, яку перетягують у вікно браузера (не всередині додаткових підпапок);

2. Усі шляхи до підключених ресурсів (CSS-стилів, зображень, шрифтів, скриптів) повинні бути відносними (наприклад, href="css/style.css" або src="assets/images/logo.svg"), а не абсолютними локальними шляхами файлової системи (як-от D:/...);
3. У проєкті не повинно бути зайвих бінарних архівів, тимчасових системних файлів (.DS_Store, Thumbs.db) чи незакритих текстових нотаток;
4. Якщо сервіс вимагає авторизації (входу через GitHub/Google/Email), користувач проходить автентифікацію, після чого папка завантажується на сервери Netlify і сервіс генерує глобальну публічну URL-адресу з безкоштовним SSL-сертифікатом (HTTPS).

12. Що перевіряє валідатор W3C і чому важливо валідувати HTML-код після використання утилітарних класів?

Валідатор W3C (World Wide Web Consortium Nu HTML Checker) перевіряє синтаксичну та структурну коректність HTML-коду відповідно до офіційних стандартів W3C і WHATWG. Він виявляє:
• Пропущені або незакриті теги, порушення ієрархії вкладеності (наприклад, розміщення блочного всередині строкового);
• Відсутність обов'язкових атрибутів (alt у <img>, lang у <html>, charset у <head>);
• Дублювання унікальних ідентифікаторів id="...";
• Використання застарілих або неіснуючих тегів та атрибутів.
Чому це критично після використання Tailwind CSS:
При активному додаванні великої кількості утилітарних класів (class="...") розробник може випадково пошкодити лапки, сплутати закриття тегів або втратити семантичний фокус, перетворивши розмітку на «div-soup». Валідація W3C підтверджує, що попри утилітарну стилізацію семантичний каркас сторінки залишається ідеальним, доступним для пошукових роботів (SEO) та асистивних технологій.

13. Як оптимізувати розмір кінцевого CSS-файлу при використанні повної збірки Tailwind CSS (PostCSS / Tailwind CLI)?

Для промислового середовища розмір стилів Tailwind мінімізується за допомогою офіційного механізму очищення (Purging / Tree-shaking), інтегрованого в рушій Tailwind CLI / PostCSS:
1. Конфігурація сканування контенту: у файлі tailwind.config.js у масиві content задаються точні патерни всіх вихідних файлів проєкту:
content: ['./*.html', './src/**/*.{js,ts,jsx,tsx}']
2. Механізм Purge / Tree-shaking: під час складання рушій сканує всі зазначені файли регулярними виразами, фіксує лише ті назви класів, які реально зустрічаються у розмітці та скриптах, і видаляє абсолютно всі невикористані класи зі стандартної багатомегабайтної бази;
3. CSS-мініфікація: запуск Tailwind CLI з прапорцем --minify додатково видаляє всі пробіли, коментарі та скорочує кольорові коди;
4. Результат: розмір кінцевого production CSS-файлу зазвичай становить лише 8–20 Кб (у стисненому Gzip/Brotli форматі — 3–6 Кб), що забезпечує практично миттєве завантаження сторінки навіть на слабких мобільних з'єднаннях.

Розділ 6. Висновки

У ході виконання лабораторної роботи №6 було детально вивчено та практично застосовано сучасну методологію верстки інтерфейсів Utility-First за допомогою фреймворку Tailwind CSS на

базі вебпроєкту «ChargeUA» (веб-помічник та пошуковик зарядних станцій). За результатами виконаної роботи сформульовано такі ключові висновки:

1. Ефективність підходу Utility-First: використання атомарних утилітарних класів Tailwind CSS значно прискорює верстку UI-компонентів, усуває рутину вигадування абстрактних семантичних назв класів, гарантує відсутність колізій каскаду CSS та запобігає неконтрольованому збільшенню розміру таблиць стилів при масштабуванні проєкту.

2. Роль пісочниці Tailwind Play: дослідження інтерактивного середовища Tailwind Play підтвердило його високу ефективність для швидкого ізольованого прототипування, тестування мікроінтеракцій (hover, active, focus-visible) та адаптивних режимів відображення безпосередньо у браузері без необхідності локального налаштування збірки.

3. Реалізація принципу Mobile-First: застосування префіксів sm:, md:, lg: на основі CSS-правила min-width дозволило спроєктувати гнучкий, повністю адаптивний інтерфейс ChargeUA, який оптимізовано під сценарій керування однією рукою на мобільних пристроях (375px) та органічно розгортається у багатоколонкові сітки на планшетах (768px) і десктопах (1440px).

4. Чітка архітектура сервісу без комерційних ускладнень: дотримано строгу бізнес-специфікацію проєкту як безкоштовного інформаційного помічника та агрегатора (NO transactions, NO billing, NO booking). Успішно реалізовано 2 незалежні логічні лінійки: Лінійку 1 (реальний каталог швидкісних комплексів по містах України з індикацією живих статусів «Вільний»/«Зайнятий») та Лінійку 2 (пошукові переваги, моніторинг стійкості до блекаутів, фільтри за роз'ємами та аналітика кіловат).

5. Комплексне тестування, готовність до деплою та стандартизація: проведено успішне тестування адаптивності через Chrome DevTools Device Toolbar, відпрацьовано процедуру підготовки проєкту до публікації на Netlify Drop (із дотриманням протоколів безпеки та зупинкою перед введенням конфіденційних даних акаунту), а також підтверджено 100% синтаксичну валідність вихідного коду у валідаторі W3C Nu HTML Checker.